PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO DE JANEIRO
Pós-Graduação *Inteligência Artificial Generativa & Large Language Models*

CARLOS FREDERICO COELHO LISBOA

AUDITORIA DE PROJETOS POWER BI

Rio de Janeiro
2026

CARLOS FREDERICO COELHO LISBOA

AUDITORIA DE PROJETOS POWER BI

Trabalho apresentado ao curso GenAI e LLM MASTER da Pontifícia Universidade Católica do Rio de Janeiro como pré-requisito para conclusão de curso e obtenção de crédito na disciplina "Projetos de Sistemas Inteligentes de Apoio à Decisão".

Orientadora: Evelyn C. S. Batista

Rio de Janeiro

2026

LISTA DE TABELAS

Tabela 1 O sinal que cada regra lê e a forma da afirmação 8
Tabela 2 Pilha tecnológica da ferramenta 9

LISTA DE FIGURAS

Figura 1 Fluxo da auditoria 9
Figura 2 Instrução de sistema enviada ao modelo de linguagem 11

SUMÁRIO

1 Método e arquitetura 5
1.1 Visão geral 5
1.2 Leitura do projeto PBIP 5
1.3 Critério de detectabilidade 6
1.4 As regras 7
1.5 Fluxo da auditoria 9
1.6 Base de documentação e recuperação 10
1.7 Geração com citação obrigatória 11
1.8 Conjunto de avaliação 12
Referências 13

1 MÉTODO E ARQUITETURA

Este capítulo descreve a ferramenta desenvolvida e as decisões de método que a sustentam. A apresentação segue o caminho de um projeto auditado: a leitura do arquivo, a detecção dos problemas, a recuperação da documentação que os fundamenta e a redação da explicação pelo modelo de linguagem. As escolhas foram registradas, ao longo do projeto, em registros de decisão de arquitetura (*Architecture Decision Records*, ADR), citados aqui pelo número quando a decisão vem de um deles.

1.1 VISÃO GERAL

A ferramenta recebe um projeto do Power BI Desktop salvo no formato PBIP (*Power BI Project*) e devolve uma lista de achados, cada um com explicação, recomendação e a passagem da documentação oficial que o fundamenta. Ela se organiza em três camadas. A primeira lê o modelo semântico do projeto e o converte numa representação interna normalizada. A segunda aplica regras determinísticas sobre essa representação e produz os achados. A terceira recupera, para cada achado, trechos da documentação do Microsoft Learn e pede a um modelo de linguagem executado localmente que explique o problema e recomende a correção, citando os trechos.

A divisão de responsabilidades entre a segunda e a terceira camadas é a decisão central do projeto (ADR-003): **as regras determinísticas são a única fonte de achados**. O modelo de linguagem não decide se um problema existe; ele apenas explica um problema que uma regra já detectou. Essa separação tem duas consequências. A precisão e a revocação da auditoria passam a depender só das regras, que são reprodutíveis e podem ser medidas contra um gabarito, sem a variação própria da geração. E o modelo de linguagem, cujo erro é mais difícil de controlar, fica restrito à parte em que um erro custa menos: uma explicação ruim degrada a redação, mas não cria nem apaga um achado.

1.2 LEITURA DO PROJETO PBIP

Um projeto PBIP é uma pasta que separa o relatório do modelo semântico. Na pasta do modelo, o arquivo `definition.pbism` declara em que formato a definição está gravada: na versão 1.0, obrigatoriamente em TMSL (*Tabular Model Scripting Language*), no arquivo `model.bim`; a partir da versão 4.0, em TMSL ou em TMDL (*Tabular Model Definition Language*), numa pasta `definition` (Microsoft, 2026). A ferramenta lê apenas o `model.bim` (ADR-001). O TMSL é um documento JSON que a biblioteca padrão do Python lê sem dependências, enquanto o TMDL não tinha, à época da decisão, analisador oficial em Python, e salvar em TMDL ainda era recurso em pré-visualização, de conversão irreversível. O formato fica

atrás de uma única função de leitura, de modo que acrescentar o TMDL no futuro não exige reescrever as regras.

O analisador converte o `model.bim` num modelo interno validado com Pydantic, com tabelas, colunas, medidas, relacionamentos, hierarquias, ordenações, papéis de segurança em nível de linha e as `variations` que ligam uma coluna de data à tabela de data automática criada pelo Power BI. Uma particularidade do TMSL é guardar expressões ora como texto, ora como lista de linhas; o analisador normaliza os dois casos.

As expressões DAX (*Data Analysis Expressions*) não são interpretadas sintaticamente, o que exigiria um analisador completo da linguagem, fora do orçamento do projeto. Elas passam por um analisador léxico próprio, que separa comentários, cadeias, números, nomes de função, referências a colunas e medidas e operadores. A afirmação de que o analisador léxico basta foi medida, não suposta: no modelo do estudo de caso (P8), as 137 expressões DAX foram transformadas em tokens sem nenhum token desconhecido, e as 105 expressões de autoria do modelador ficaram cobertas pela varredura, sem lacunas. Quando uma expressão não é coberta, a ferramenta declara a lacuna no resultado em vez de silenciar.

1.3 CRITÉRIO DE DETECTABILIDADE

O planejamento inicial previa entre 20 e 25 regras, com uma cota por categoria. A cota foi substituída, ainda no início do projeto, por um critério que pergunta outra coisa: não quantas regras cabem em cada categoria, mas quais problemas podem ser identificados de forma confiável a partir do arquivo. O critério de detectabilidade foi construído em três etapas, cada uma motivada por uma falha concreta, e é aplicado **antes** de escrever o código de uma regra.

O primeiro teste é a **decidibilidade no arquivo**: o problema precisa ser observável no `model.bim`. Ele ordena as regras em grupos de prioridade. As estruturais, que leem propriedades explícitas do JSON, vêm primeiro; as de DAX com padrão textual inequívoco, depois; as de DAX dependentes de contexto de avaliação ficaram fora, porque sem analisador sintático a heurística erra com frequência; e as de Power Query M ficaram fora do produto mínimo.

O segundo teste é a **âncora citável**: uma regra só entra se houver uma passagem do Microsoft Learn que a sustente, lida e transcrita antes do código. Sem essa passagem, a regra não pode cumprir a exigência de que toda explicação cite a documentação. Aplicado ao conjunto estrutural, o teste eliminou três das nove regras propostas. Uma delas marcaria como defeito o uso de tabelas calculadas em DAX, mas a página que a sustentaria recomenda justamente criar tabelas de data com as funções `CALENDAR` e `CALENDARAUTO` (Microsoft, 2025). As outras duas não tinham passagem que as fundamentasse.

O terceiro teste é a **suficiência da evidência**. Ele nasceu de uma regra que passava nos dois primeiros e ainda assim errava: a MOD-005, que deveria apontar a dimensão de data

não marcada como tabela de data, lia a ausência de uma propriedade como prova de que a marcação não fora feita. Mas o Power BI Desktop nunca grava essa propriedade, de modo que a ausência era compatível com os dois casos e a regra marcaria também a dimensão corretamente configurada. O teste exige que a regra enumere por escrito, antes do código, (a) todas as formas de escrever a condição procurada, (b) todas as coisas que se parecem com o sinal sem sê-lo e (c) todos os lugares do arquivo onde o sinal pode estar. Só então a regra declara se afirma pela **presença** ou pela **ausência** de algo; afirmar por ausência exige as três enumerações completas e a garantia de que a presença seria gravada no caso contrário. Faltando qualquer uma, a regra se cala e o falso negativo é declarado. A MOD-005 foi reescrita: passou a usar como prova a `variations` da coluna de data apontando para uma tabela de data automática, que o Power BI remove quando a tabela é marcada. Com o tempo automático desligado, ela se cala.

O caso que melhor ilustra o critério é uma regra rejeitada. A candidata PERF-005 apontaria colunas sem uso no modelo e era a de maior rendimento esperado do grupo, com âncora já transcrita: a página de redução de dados recomenda remover colunas que não sirvam nem ao relatório nem à estrutura do modelo (Microsoft, 2024a). A cláusula (c) a derrubou. O uso de uma coluna pelo relatório mora na camada de visuais, que a ferramenta não lê, e a regra afirmaria uma ausência sobre um domínio que não observa. A medição no P8 confirmou o problema: dos 33 achados que a regra emitiria, 5 eram defensáveis, 3 eram falsos positivos confirmados e os 25 restantes dependiam da camada de relatório. A precisão ficaria entre cerca de 15% e cerca de 91%, conforme o uso dessas 25 colunas, uma faixa larga demais para sustentar qualquer número diante do limiar de 0,7 adotado no projeto. A regra não foi implementada.

Por isso, a contagem final de regras deixou de ser meta e passou a ser **resultado** do método. O conjunto tem 9 regras, menos da metade do previsto, e a diferença não é lacuna de esforço: cada regra ausente é um caso em que o critério impediu um achado que a ferramenta não poderia sustentar. O argumento é o mesmo da literatura de análise estática, discutido no Capítulo 2: uma regra com precisão baixa é pior que uma regra ausente, porque compromete a confiança no relatório inteiro.

1.4 AS REGRAS

As 9 regras se dividem em três categorias. O catálogo completo, com a página do Microsoft Learn que ancora cada uma, está no Apêndice A.

As seis regras de **modelagem** leem a estrutura do modelo. A MOD-001 aponta o tempo automático de data/hora ligado, que cria uma tabela de data oculta para cada coluna de data e aumenta o tamanho do modelo (Microsoft, 2025). A MOD-002 aponta relacionamentos com filtro bidirecional, fonte de ambiguidade na propagação de filtros. A MOD-003 aponta tabelas sem nenhum relacionamento com o restante do modelo. A MOD-005, já discutida, aponta a

dimensão de data não marcada como tabela de data. A MOD-006 aponta dimensões normalizadas em floco de neve, que o esquema estrela recomenda, em geral, integrar numa única tabela (Microsoft, 2024b). A MOD-007 aponta relacionamentos um-para-um.

As duas regras de **desempenho** leem propriedades das colunas. A PERF-001 aponta colunas calculadas em DAX: o motor VertiPaq as armazena com compressão tipicamente menos eficiente que a de colunas criadas no Power Query, e elas prolongam a atualização dos dados (Microsoft, 2024a). A PERF-003 aponta colunas de ponto flutuante cuja agregação padrão é a soma.

A regra de **DAX**, DAX-001, lê o texto das expressões e aponta divisões feitas com o operador `/` quando o denominador não é constante. A âncora mudou o enunciado da regra. A versão inicial, "divisão com `/` em vez de `DIVIDE`", marcaria como defeito o que a própria documentação recomenda quando o denominador é constante: usar o operador, que evita um teste desnecessário (Microsoft, 2021). A regra implementada afirma apenas sobre denominador não constante, condição que as duas passagens da página sustentam juntas.

A Tabela 1 resume o sinal que cada regra lê no `model.bim` e se ela afirma pela presença ou pela ausência de algo. Só a MOD-003 afirma por ausência, e pode fazê-lo porque o terceiro teste se cumpre por inteiro: todos os relacionamentos de um modelo estão no próprio arquivo, de modo que uma tabela ausente de todos eles está, de fato, desconectada. Mesmo assim, como mostra o Capítulo 4, a regra erra em tabelas desconectadas de propósito, um caso que o critério não previu.

Tabela 1 – O sinal que cada regra lê e a forma da afirmação

Regra	**Sinal lido no `model.bim`**	**Afirma por**
MOD-001	Tabela de data gerada pelo produto (tempo automático)	presença
MOD-002	Relacionamento com filtro cruzado nos dois sentidos, exceto um-para-um	presença
MOD-003	Tabela ausente de todos os relacionamentos do modelo	ausência
MOD-005	`variations` da coluna de data da dimensão apontando para uma tabela automática	presença
MOD-006	Tabela no lado "um" de um relacionamento e no lado "muitos" de outro	presença
MOD-007	Relacionamento com cardinalidade "um" nas duas pontas	presença
PERF-001	Coluna calculada em DAX, exceto colunas de agrupamento e colunas de uma dimensão de data calculada em DAX	presença
PERF-003	Coluna de ponto flutuante com agregação padrão de soma	presença
DAX-001	Operador `/` com denominador não constante, por expressão	presença

Fonte: Elaborado pelo autor a partir do código das regras (`core/rules/`).

As exceções da PERF-001 também vêm da documentação: as colunas de agrupamento têm o DAX escrito pela própria interface, e as páginas sobre tempo automático e tabelas de data recomendam criar a tabela de data com `CALENDAR` ou `CALENDARAUTO` e acrescentar colunas calculadas a ela (Microsoft, 2025). Sem essas exceções, a regra recomendaria desfazer o que a fonte recomenda fazer.

1.5 FLUXO DA AUDITORIA

A auditoria é uma função que executa as etapas em sequência, sem orquestração de agentes (ADR-003), conforme a Figura 1. O fluxo é linear e um arcabouço de agentes não traria ganho demonstrável; a sequência fixa também facilita explicar e testar cada etapa isoladamente.

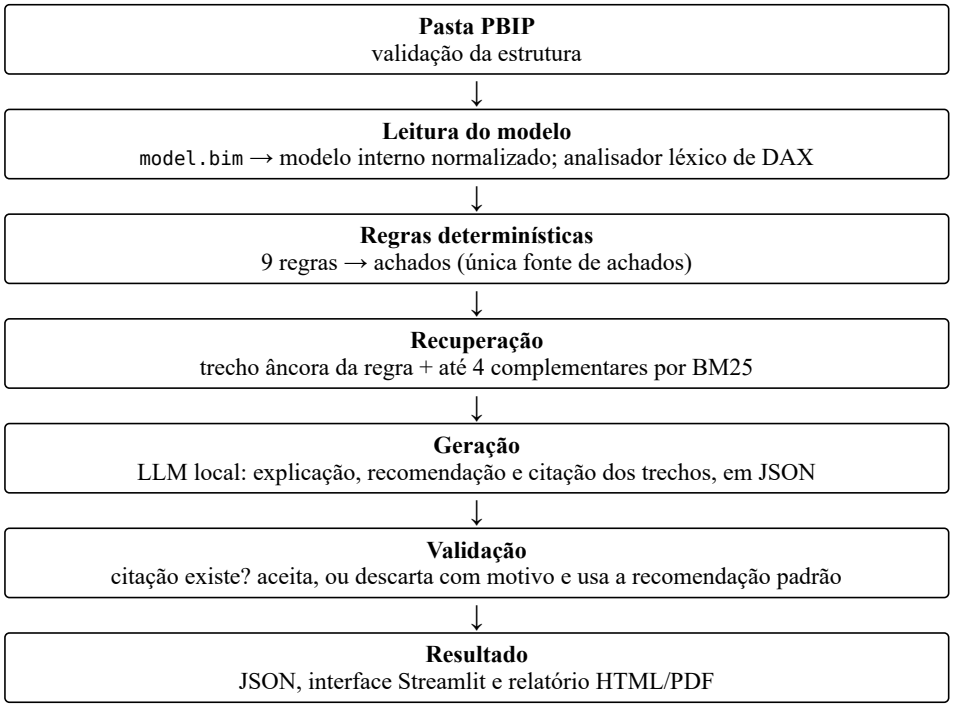


Figura 1 – Fluxo da auditoria

Fonte: Elaborado pelo autor.

As regras são executadas em ordem estável e isoladas umas das outras: se uma regra falha com erro, ela não interrompe a auditoria, e sua saída parcial é descartada inteira, para que nenhum achado venha de uma execução incompleta. Antes de chamar o modelo de linguagem, a auditoria verifica que toda regra tem gabarito de âncora e que a seção âncora existe no índice; faltando qualquer uma, ela para em vez de gerar uma explicação sem fundamento. Além da linha de comando, a ferramenta tem uma interface em Streamlit e gera um relatório em HTML e PDF.

A Tabela 2 lista as tecnologias usadas (ADR-002). A escolha seguiu duas restrições do ambiente: tudo precisava instalar no Windows 11 sem compilador C++, e o modelo de linguagem precisava caber numa GPU de 6 GB. A recuperação foi escrita sem arcabouço, como um fluxo curto e próprio, porque um arcabouço como LangChain ou LlamaIndex acrescentaria abstrações que um fluxo linear não pede e que dificultariam explicar o funcionamento.

Tabela 2 – Pilha tecnológica da ferramenta

Camada	**Escolha**
Linguagem	Python 3.11, com Pydantic para o modelo interno validado
Leitura do modelo	`json` da biblioteca padrão sobre o `model.bim`; analisador léxico de DAX próprio
Recuperação	Fluxo próprio: coleta em HTML, extração por seção, índice BM25 (`rank-bm25`)

Camada	Escolha
Modelo de linguagem	`qwen2.5:7b-instruct` quantizado em 4 bits, executado pelo Ollama na GPU local
Interface	Streamlit
Relatório	Jinja2 para o HTML; Playwright controlando o Microsoft Edge do sistema para o PDF
Testes e avaliação	pytest; pacotes próprios para o gabarito, a medição da recuperação e a rubrica da geração

Fonte: Elaborado pelo autor a partir da ADR-002.

Todo o desenvolvimento foi guiado por testes automatizados, escritos antes do código. Além dos testes de unidade, há testes que conferem artefatos contra a sua fonte: o catálogo de fontes da recuperação é regenerado e comparado com o arquivo versionado, a passagem citada por cada regra é procurada no texto extraído da base, e as tabelas de resultado desta monografia são geradas dos arquivos de resultado e comparadas com as que o texto inclui. Assim, um número que mude na origem faz a suíte falhar até que o documento seja atualizado.

1.6 BASE DE DOCUMENTAÇÃO E RECUPERAÇÃO

A base consultada pela recuperação é a documentação pública do Microsoft Learn. O plano inicial de clonar o repositório de origem da documentação não se sustentou, porque o repositório é privado. As páginas foram então coletadas em HTML a partir dos índices publicados de cada seção do site, depois de verificado que o arquivo `robots.txt` não bloqueia o caminho (ADR-004). O catálogo de fontes é gerado desses índices, e não digitado, e reúne 1.420 páginas: a seção de orientação do Power BI inteira, as referências completas de funções DAX e de Power Query M e as páginas âncora das regras. A cópia local das páginas fica fora do repositório; versionam-se o catálogo e o código de coleta. Seis artigos do SQLBI entram apenas como leitura complementar, com link, porque os termos do site não autorizam armazenar os artigos.

Cada página é extraída por seção, preservando a âncora e o caminho de títulos de cada uma, e as seções são os trechos indexados, com até 350 palavras. A recuperação tem dois níveis (ADR-002, emenda de 09/10/2026). O primeiro é fixo: a **seção âncora** da regra, a passagem transcrita quando a regra passou no segundo teste do critério, entra sempre como o trecho número 1. O segundo vem da busca, que acrescenta trechos complementares. Essa construção nasceu de uma falha observada no teste inicial do modelo de linguagem: com várias páginas no contexto, o modelo citou uma página válida, mas menos pertinente que a âncora. Com a âncora fixa, a passagem que sustenta o achado está sempre disponível, e a qualidade da busca passa a afetar só os complementares.

O método de busca foi escolhido por medição, com a régua definida antes de medir. Foram comparadas 24 combinações de fragmentação (por seção ou por janela de palavras), cabeçalho de contexto (com ou sem), tipo de busca (densa, com o modelo de vetores `bge-small-`

`en-v1.5`; lexical, com BM25 (Robertson; Zaragoza, 2009); ou híbrida, por fusão recíproca de posições (Cormack; Clarke; Büttcher, 2009)) e corpus (completo ou só orientação). A regra de decisão, fixada antes, escolhia a maior taxa de acerto entre os cinco primeiros resultados no corpus completo e, em caso de empate próximo, a combinação mais simples. Venceu a fragmentação por seção, sem cabeçalho, com BM25 sobre o corpus completo. Os números e as limitações dessa medição estão no Capítulo 4.

1.7 GERAÇÃO COM CITAÇÃO OBRIGATÓRIA

A explicação é redigida pelo `qwen2.5:7b-instruct` (Qwen Team, 2024), quantizado em 4 bits e executado localmente pelo Ollama numa GPU de 6 GB. O modelo foi escolhido num teste comparativo com a versão de 3 bilhões de parâmetros: os dois citaram corretamente nos cinco casos de teste, mas o menor errou um número que estava na própria evidência. A execução local tem uma razão prática, além do custo: o modelo semântico de um cliente não precisa sair da máquina de quem o audita. A janela de contexto foi fixada em 4.096 tokens, o máximo que cabe na memória da GPU sem descarregar camadas para a CPU, e a geração usa temperatura 0 e semente fixa.

O pedido ao modelo tem duas partes. A instrução diz que o problema já foi detectado por uma regra determinística e que o modelo não decide se ele existe; que deve usar só os trechos fornecidos; citar de 1 a 3 trechos pelo número; e responder em português do Brasil, de forma curta e acionável. A mensagem traz a regra, o objeto afetado, a evidência do achado e os trechos numerados, em inglês, a língua do corpus. A resposta tem de seguir um esquema JSON imposto pelo Ollama, com explicação, recomendação e a lista de citações. A Figura 2 reproduz a instrução de sistema, igual em todos os achados. A citação é pedida pelo número do trecho, e não pelo endereço da página, porque um modelo de 7 bilhões de parâmetros copia mal cadeias longas; o código traduz o número para página e seção.

```
Você é um auditor de modelos semânticos do Power BI.
Uma regra determinística JÁ detectou o problema descrito pelo usuário. Você NÃO decide se o problema existe: sua tarefa é explicá-lo e recomendar a correção.
Use somente as informações do achado e dos trechos de documentação numerados. Não invente números, nomes de objetos nem fatos.
Responda em português do Brasil, em JSON, com:
- "explicacao": por que isso é um problema neste modelo;
- "recomendacao": o que fazer, de forma acionável;
- "citacoes": de 1 a 3 números dos trechos que sustentam a explicação.
Explicação e recomendação juntas devem ter no máximo 120 palavras.
```

Figura 2 – Instrução de sistema enviada ao modelo de linguagem
Fonte: Código da ferramenta (`core/analise/prompt.py`).

A validação é feita por código. Se a resposta não é JSON válido, se um campo vem vazio, se o número de citações sai do intervalo ou se algum número citado não existe no contexto, a

resposta é descartada, sem nova tentativa, e o motivo é registrado. O achado continua no resultado com a recomendação padrão da regra e a marca "sem fundamentação gerada": o descarte nunca apaga um achado. Uma resposta aceita passa ainda por uma conferência de superfície, que verifica se números e nomes de objetos citados na explicação aparecem no achado ou nos trechos. Por ser uma heurística ainda não calibrada, ela registra divergências, mas não descarta. O Capítulo 4 mostra que essa conferência não mede o que a leitura humana chama de fidelidade.

As respostas podem ser guardadas num cache, indexado pelo pedido completo, pelo modelo e pelos parâmetros de geração. O cache torna a auditoria repetível em segundos com textos idênticos, o que é necessário porque a temperatura 0 não torna a geração do Ollama determinística: entre duas execuções, os totais se repetem, mas os textos e algumas citações variam. Ele vem desligado por padrão e, quando usado, é indicado no resultado e no relatório.

1.8 CONJUNTO DE AVALIAÇÃO

As métricas vêm de sete projetos públicos do repositório de exemplos do Power BI Desktop mantido pela Microsoft, sob licença MIT, convertidos manualmente para PBIP em TMSL (ADR-005). Eles cobrem sete domínios: vendas, despesas corporativas, recursos humanos, marketing, varejo, cadeia de suprimentos e funil comercial. Cinco vêm da revisão de 2026 dos exemplos e dois de amostras mais antigas, uma mistura que evita um conjunto uniformemente bom ou ruim. Por serem públicos, qualquer leitor pode refazer a avaliação com os mesmos arquivos. Esses projetos são chamados de P1 a P7.

Um oitavo projeto, P8, é um modelo de autoria própria sobre vendas de varejo online. Ele chegou a integrar o conjunto de métricas, mas saiu dele porque não pode ser reproduzido por terceiros, e passou a estudo de caso: mostra a ferramenta agindo em profundidade sobre um modelo com problemas reais, a começar pelo tempo automático ligado ao lado de uma dimensão de data própria. Nenhuma métrica do Capítulo 4 depende do P8.

O gabarito das regras foi anotado pelo autor. Para cada regra, um gerador independente da lógica da regra lista todos os objetos que poderiam violá-la — por exemplo, todas as tabelas, para a MOD-003, ou todas as colunas calculadas, para a PERF-001 —, acompanhados de fatos neutros sobre cada um. A anotação foi cega: a planilha não informava quais objetos a ferramenta marcara, e o anotador julgava cada um contra a pergunta da âncora, com os vereditos "viola", "não viola" ou "indeterminado", este último reservado aos casos em que o modelo não traz a informação necessária. Nenhuma regra mudou durante a anotação, e a versão medida foi congelada numa marca de versão do repositório no commit que a encerrou. O protocolo previa criar a marca antes de anotar; ela foi criada depois, mas sobre o mesmo código. Correções de regra feitas a partir daí são relatadas como análise de erros, medidas contra o mesmo gabarito, e não como resultado.

REFERÊNCIAS

CORMACK, Gordon V.; CLARKE, Charles L. A.; BÜTTCHER, Stefan. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. **Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '09), Boston**, p. 758–759, 2009.

MICROSOFT. **DIVIDE function vs divide operator (/) in DAX**. 2021. Disponível em: <https://learn.microsoft.com/en-us/dax/best-practices/dax-divide-function-operator>. Acesso em: 8 out. 2026.

MICROSOFT. **Data reduction techniques for Import modeling**. 2024a. Disponível em: <https://learn.microsoft.com/en-us/power-bi/guidance/import-modeling-data-reduction>. Acesso em: 15 out. 2026.

MICROSOFT. **Understand star schema and the importance for Power BI**. 2024b. Disponível em: <https://learn.microsoft.com/en-us/power-bi/guidance/star-schema>. Acesso em: 15 out. 2026.

MICROSOFT. **Auto date/time guidance in Power BI Desktop**. 2025. Disponível em: <https://learn.microsoft.com/en-us/power-bi/guidance/auto-date-time>. Acesso em: 15 out. 2026.

MICROSOFT. **Power BI Desktop project semantic model folder**. 2026. Disponível em: <https://learn.microsoft.com/en-us/power-bi/developer/projects/projects-dataset>. Acesso em: 15 out. 2026.

QWEN TEAM. Qwen2.5 Technical Report. **arXiv preprint arXiv:2412.15115**, 2024.

ROBERTSON, Stephen; ZARAGOZA, Hugo. The Probabilistic Relevance Framework: BM25 and Beyond. **Foundations and Trends in Information Retrieval**, v. 3, n. 4, p. 333–389, 2009.